

Homework 2

by Petros Efraim Koukios, and Alex Rysström
Group 68

February 4, 2026

Question 4: The 8-queens problem (40 points)

In this homework, parallel programming using OpenMP was used to solve the 8-queens problem. The problem involves placing 8 queens on a chess board in such a way that they cannot attack each other (not in the same column, row, or diagonal). This yields a total of 92 solutions. As will be explained later in the report, the program was run on up to 8 processors and the speedup was calculated as well. This report will cover some of the main ideas used in the solutions as well as a performance evaluation.

Firstly, a function called *is_safe* was written. This looks at all previous columns to see if a queen can be placed at a certain coordinate (row, col) without being attacked, as can be seen in the following code snippet:

```
// Check whether placing a queen at (row, col) is safe
// q_placed_row[i] = row index of queen in column i
int is_safe(int q_placed_row[], int col, int row) {
    for (int i = 0; i < col; i++) {
        // Row check
        if (q_placed_row[i] == row) return 0;
        // Diagonal check
        if (abs(i - col) == abs(q_placed_row[i] - row)) return 0;
    }
    return 1;
}
```

Subsequently, the recursive *solve_queens* was written to place queens column by column. The function for its specific column, tries all rows, checks if any position is valid, and places the queen in a copy of the board. Then the function is called again recursively to solve further. Only the first 2 columns create separate tasks, whereas deeper recursion is done sequentially to avoid excessive thread overhead. If all columns are filled, the solution count is incremented atomically. The function can be seen as shown:

```
// Recursive function to place queens column by column
void solve_queens(int q_placed_row[], int col) {
    // Base case
    if (col == 8) {
        #pragma omp atomic
        solution_count++;
        return;
    }

    // Try all rows in current column
    for (int row = 0; row < 8; row++) {
        // Check if position is valid
        if (is_safe(q_placed_row, col, row)) {
```

```

        // Create local copy of the board
        int new_q_placed_row[8];
        for (int i = 0; i < col; i++)
            new_q_placed_row[i] = q_placed_row[i];

        new_q_placed_row[col] = row;

        // Create tasks only for upper levels of recursion
        if (col < 2) {
            #pragma omp task firstprivate(new_q_placed_row, col)
            // This call becomes a separate task
            solve_queens(new_q_placed_row, col + 1);
        } else {
            // Deeper recursion done sequentially to reduce overhead
            solve_queens(new_q_placed_row, col + 1);
        }
    }
}

// Wait for all child tasks to complete before returning
#pragma omp taskwait
}

```

Lastly, in the main function, `omp_get_wtime()` was used to calculate the parallel execution time of the program run on a varying number of threads. The number of threads, the median execution time and the speedup were also labelled, calculated and output. The methodology will be described further in the performance evaluation section below.

Performance Evaluation

We evaluated the performance of the parallel 8-queens solver by varying the number of OpenMP threads from 1 to 8 and measuring the parallel computation phase of the program. To overcome timer resolution limitations and reduce noise from operating system scheduling, we computed each reported value as follows: for one measurement, the solver was executed 100 times and the mean execution time was recorded. This process was repeated 100 times, and the median of these mean values was used as the final execution time. Speedup was calculated as the ratio of the median single-thread execution time to the median execution time obtained with multiple threads.

Threads	Median Execution Time (s)	Speedup
1	0.00054	1.000
2	0.00031	1.742
3	0.00030	1.800
4	0.00025	2.160
5	0.00025	2.160
6	0.00026	2.077
7	0.00026	2.077
8	0.00026	2.077

Table 1: Performance of the OpenMP task-based 8-Queens solver using a median-of-means timing method.

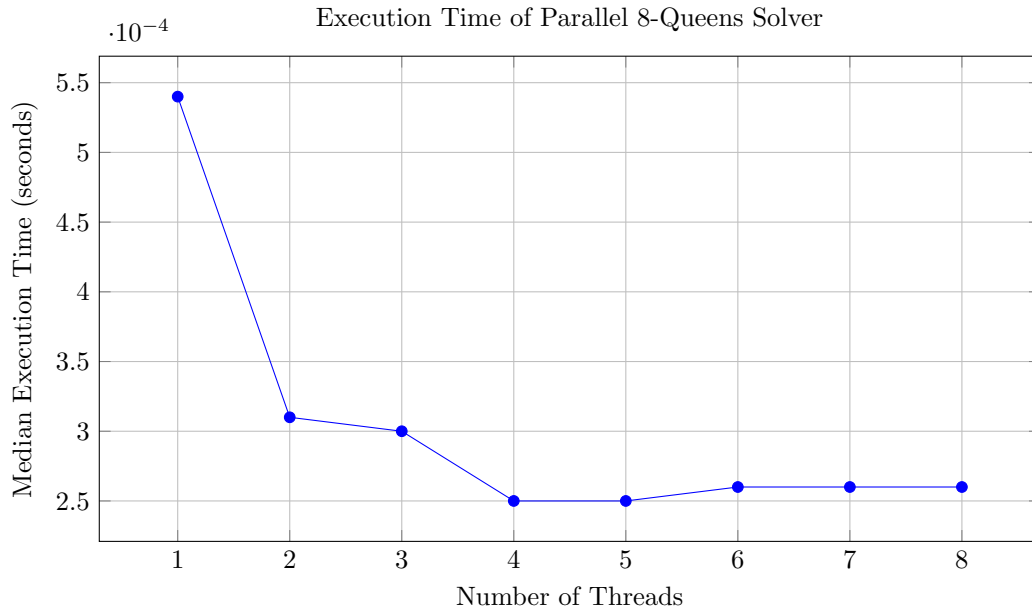


Figure 1: Median execution time versus thread count.

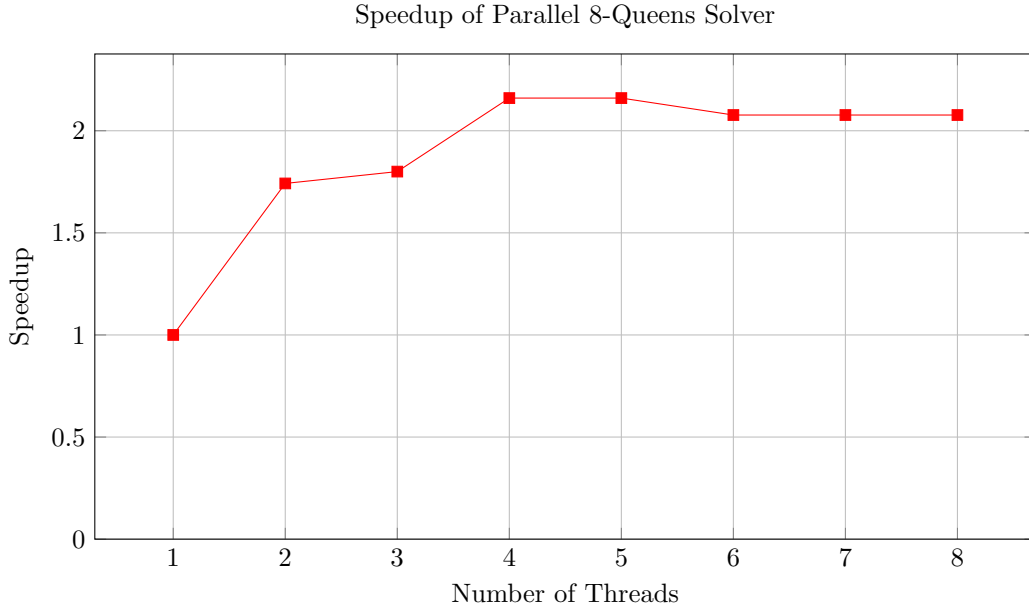


Figure 2: Speedup achieved by increasing the number of OpenMP threads.

The results show that performance improves greatly when increasing the number of threads from one to four, reaching a maximum speedup of around $2.16\times$. Beyond four threads, performance starts to plateau and slightly degrade. This indicates that the available parallel work in the 8-queens problem has been saturated. Since the search tree is relatively small and contains only 92 valid solutions, the amount of independent work is limited.

The more threads that are added, the overhead of thread and task creation, scheduling, and synchronization becomes more and more significant compared to useful computation, thus performance degrades. According to Amdahl's Law, the sequential part of the program limits the maximum achievable speedup overall, which we can clearly see here as the curve flattens beyond four threads. These results demonstrate the practical limits of parallelism for small combinatorial search problems.